

딥러닝 프레임워크

피부질환 분류모델

구승울 | 김민석 | 김영준

목차

- 01 ————— 피부질환 분류 딥러닝 알고리즘 설계
- 02 ————— 피부질환 분류 타입
- 03 ————— 데이터 셋과 예측 정확도
- 04 ————— 코드구현단계 - 최적화 기법
- 05 ————— 시연 결과
- 06 ————— 예측 예제

×
×
×



01

피부질환 분류 딥러닝 알고리즘 설계

피부 질환 분류 타입



여드름

학습 데이터: 2202개
테스트 데이터: 550개



습진

학습 데이터: 1796개
테스트 데이터: 457개



피부암

학습 데이터: 2053개
테스트 데이터: 499개



정상 피부

학습 데이터: 1987개
테스트 데이터: 639개

kaggle™

데이터 수집장소: kaggle 데이터셋
Skin disease dataset에서 이미지 데이터 활용

역할 분담



구승율

AI 학부 2학년

데이터 수집,
데이터 전처리



김민석

AI 학부 2학년

데이터 정제, 모델
학습 및 실행



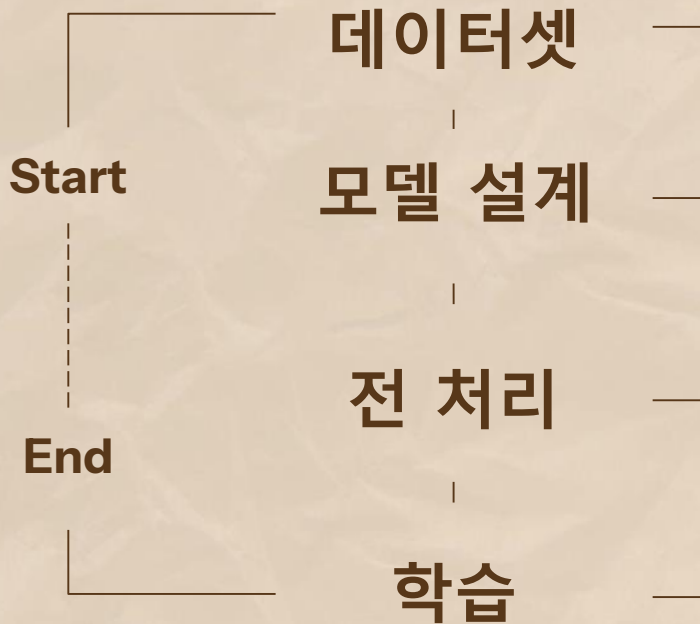
김영준

AI 학부 2학년

데이터셋, 모델 설계

×
×
×

예측 코드 구현 단계



데이터셋 코드

```
class CustomImageDataset(Dataset):
    def read_data_set(self):

        all_img_files = []
        all_labels = []

        class_names = os.walk(self.data_set_path).__next__()[1]

        for index, class_name in enumerate(class_names):
            label = index
            img_dir = os.path.join(self.data_set_path, class_name)
            img_files = os.walk(img_dir).__next__()[2]

            for img_file in img_files:
                img_file = os.path.join(img_dir, img_file)
                img = Image.open(img_file)
                if img is not None:
                    all_img_files.append(img_file)
                    all_labels.append(label)

        return all_img_files, all_labels, len(all_img_files), len(class_names)
```

} 데이터 셋 클래스를 정의하여 이미지 파일 경로와 레이블을 저장할 빈 리스트를 생성하고 주어진 경로에서 class_names 에 저장

} 각 클래스에 대해 인덱스와 클래스 이름을 매핑하고, 해당 클래스의 디렉토리에서 이미지 파일목록을 가져온다.

} 각 이미지 파일에 대해 이미지를 열어보고, 이미지가 존재하면 파일 경로와 레이블을 리스트에 추가한다.



데이터셋 코드

```
def __init__(self, data_set_path, transforms=None):
    self.data_set_path = data_set_path
    self.image_files_path, self.labels, self.length, self.num_classes = self.read_data_set()
    self.transforms = transforms

def __getitem__(self, index):
    image = Image.open(self.image_files_path[index])
    image = image.convert("RGB")

    if self.transforms is not None:
        image = self.transforms(image)

    return {'image': image, 'label': self.labels[index]}

def __len__(self):
    return self.length
```

데이터셋의 이미지를 객체로 생성하여
클래스의 속성으로 설정

이미지 데이터를 열고 RGB형식으로 변환

모델 설계

```
class Convbox(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(Convbox, self).__init__()
        # 첫 번째 컨볼루션 레이어
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        # 두 번째 컨볼루션 레이어
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # 네트워크의 깊이를 늘릴 때 발생하는 그래디언트 소실 문제를 완화(앞선의 입력을 더해줌으로써)
        self.storage = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.storage = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        storage = self.storage(x) # 스킵 연결 부분을 먼저 계산
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        if storage.size(2) != out.size(2) or storage.size(3) != out.size(3):
            storage = F.interpolate(storage, size=(out.size(2), out.size(3)), mode='nearest')
        out += storage # 스킵 연결을 더함
        out = self.relu(out)
        return out
```

클래스에서 입력, 출력 채널 수와 스트라이드를 매개변수로 설정

3x3 크기의 필터를 설정하여 출력 채널에 대한 배치 정규화 레이어 생성

Forward 함수를 정의하여 정방향계산을 통해 컨볼루션 레이어 통과 후 연결과 출력의 크기를 맞춘 후 ReLU 함수 적용

모델 설계

```
class CustomSkinClassification(nn.Module):
    def __init__(self, num_classes, dropout_rate=0.3):
        super(CustomSkinClassification, self).__init__()

        # 초기 컨볼루션 레이어
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.dropout = nn.Dropout2d(p=dropout_rate)

        # Convbox Layer 1
        self.layer1 = nn.Sequential(
            Convbox(64, 64),
            Convbox(64, 64),
            Convbox(64, 64)
        )

        # Convbox Layer 2
        self.layer2 = nn.Sequential(
            Convbox(64, 128, stride=2),
            Convbox(128, 128),
            Convbox(128, 128),
            Convbox(128, 128, dropout_rate=dropout_rate)
        )

        # Convbox Layer 3
        self.layer3 = nn.Sequential(
            Convbox(128, 256, stride=2),
            Convbox(256, 256),
            Convbox(256, 256),
            Convbox(256, 256),
            Convbox(256, 256),
            Convbox(256, 256, dropout_rate=dropout_rate)
        )
```

초기 레이어는 입력 채널이 3, 출력 채널이 64인 7x7 크기의 컨볼루션 레이어(초반에 사이즈를 줄이기 위하여 큰 커널 적용)를 생성하고 BatchNorm 과 ReLU 를 적용 후 3x3 크기의 stride 2로 Maxpooling을 진행함으로써 가로 세로 크기를 절반으로 줄임

각각의 레이어를 일일이 나열하기엔 그 수가 너무 많아서 효율적 표현 및 코드 가독성을 위하여 레이어를 크게 4개 층으로 분류

각각의 레이어 안에 Convbox를 적절하게 배치함으로써 입력값에 대한 특징을 최대한 추출하려고 노력함

각 레이어의 맨 끝에 과적합 방지 위해 dropout을 설정함으로써 모델이 특정 뉴런에 의존하는 것을 방지, 모든 뉴런이 입력 데이터의 특징을 학습하는데 도움이 되도록 함



모델 설계

```
# Convbox Layer 4
self.layer4 = nn.Sequential(
    Convbox(256, 512, stride=2),
    Convbox(512, 512),
    Convbox(512, 512, dropout_rate=dropout_rate)
)

# Global Average Pooling 및 Fully Connected Layer
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.fc = nn.Linear(512, num_classes)

def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)

    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)

    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.fc(x)

    return x
```

} **AdaptiveAvgPool2d** 는 입력 텐서의 크기에 관계없이 출력 텐서의 크기를 일정하게 유지할 수 있도록 함.

이는 입력 텐서의 각 채널에 대해 평균 풀링을 수행하여, 각 채널의 평균값을 계산함.

고차원의 피쳐맵을 낮은 차원의 벡터로 변환하기 위해 사용함

} 입력값은 차례대로 이 순서를 지나가면서 특징 추출이 된 후 **flatten** 되어 1차원 벡터가 된 후 **fully connected layer**를 통해 클래스 분류를 하게됨

전 처리

```
hyper_param_epoch = 20
hyper_param_batch = 64
hyper_param_learning_rate = 0.001

resize_train_mean=[0.485, 0.456, 0.406]
resize_train_std=[0.229, 0.224, 0.225]

resize_test_mean=[0.485, 0.456, 0.406]
resize_test_std=[0.229, 0.224, 0.225]

transform_train = transforms.Compose([
    transforms.Resize((256, 256)), # 이미지 resize
    transforms.RandomCrop(224), # 이미지를 랜덤으로 크롭
    transforms.ToTensor(),
    transforms.Normalize(resize_train_mean, resize_train_std)
])

transform_test = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224), # 이미지를 랜덤으로 크롭
    transforms.ToTensor(),
    transforms.Normalize(resize_test_mean, resize_test_std)
])

train_data_set = CustomImageDataset(data_set_path="/content/drive/MyDrive/data/version1/train", transforms=transform_train)
test_data_set = CustomImageDataset(data_set_path="/content/drive/MyDrive/data/version1/test", transforms=transform_test)

train_loader = DataLoader(train_data_set, batch_size=hyper_param_batch, shuffle=True)
test_loader = DataLoader(test_data_set, batch_size=hyper_param_batch, shuffle=False)
```

학습에 사용될 에폭 수를 20, 배치 크기를 64로
설정하고 학습률 지정

훈련과 테스트 이미지를 정규화

Transform으로 이미지 크기를 256x 256으로 조정하고
이미지 데이터셋 파일을 불러온 다음, 배치단위의
데이터셋을 불러와 데이터 로더를 생성

모델 과적합 수정 과정

에포크 설계

에포크를 5단위로 증가시키며 수행, 20 에포크 미만에서 적은 에포크 횟수로 인해 학습률 저하, 30이상 에포크에서 20과 비슷한 정확도를 보였지만 학습 시간이 오래 걸리며 과대적합이 나타남.

하이퍼 파라미터 튜닝

배치 사이즈는 32, 64, 128순으로 실험하였고 64에 최적의 학습률을 보이며 128이상부터는 메모리 부족으로 에러가 발생하였음.

Batch_normalize

훈련데이터와 테스트데이터의 배치를 정규화하여 데이터 분포를 통일 시켜 과적합을 방지 하였음.

데이터 정제

모델 학습에 통일성을 부여하기 위하여 클래스당 이미지의 개수는 약 2500개로 분류하였으며, 8대2비율(train:80%, test:20%)로 이미지 개수를 조정하였음.

학습

```
# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(custom_model.parameters(), lr=hyper_param_learning_rate)

train_losses = []
test_losses = []
train_accuracies = []
test_accuracies = []

for e in range(hyper_param_epoch):
    custom_model.train()
    correct_train = 0
    total_train = 0

    for i_batch, item in enumerate(train_loader):
        images = item['image'].to(device)
        labels = item['label'].to(device)

        # Forward pass
        outputs = custom_model(images)
        loss = criterion(outputs, labels)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        _, predicted_train = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

    if (i_batch + 1) % hyper_param_batch == 0:
        print('에포크 [{0}/{0}], 배치 [{0}/{0}], 훈련 손실: {:.4f}'
              .format(e + 1, hyper_param_epoch, i_batch + 1, len(train_loader), loss.item()))

# 훈련 정확도 기록
train_accuracy = 100 * correct_train / total_train
train_accuracies.append(train_accuracy)
print('훈련데이터 {}개에 대한 정확도: {} %'.format(total_train, train_accuracy))
```

손실 및 정확도를 기록할 리스트 초기화:
훈련 및 테스트 에폭마다 손실과 정확도를
기록할 빈 리스트를 초기화

배치 이미지를 가져와 Train 데이터에 대한
학습을 설정하여 모델에 대한 예측을
수행하고 손실 값을 계산

손실에 대한 역전파를 실행하고
optimizer를 통해 모델의 가중치를
업데이트 한 후 학습된 모델에 대해
정확도를 계산하고 정확도와 손실을 출력

학습

```
for item in test_loader:
    images = item['image'].to(device)
    labels = item['label'].to(device)
    outputs = custom_model(images)
    _, predicted_test = torch.max(outputs.data, 1)
    total_test += len(labels)
    correct_test += (predicted_test == labels).sum().item()

# 테스트 정확도 기록
test_accuracy = 100 * correct_test / total_test
test_accuracies.append(test_accuracy)
print('테스트 데이터 {}개에 대한 정확도: {} %'.format(total_test, test_accuracy))

plt.plot(train_accuracies, label='Train Accuracy')
plt.plot(test_accuracies, label='Test Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.show()
```

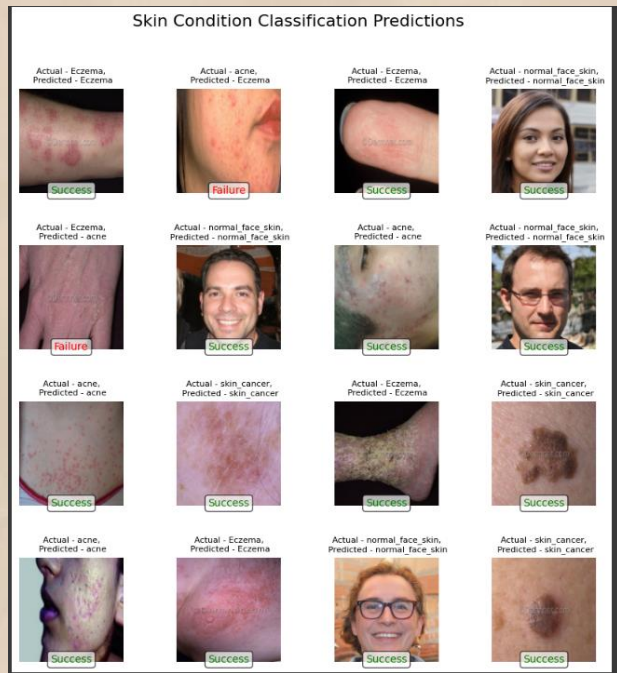
모델을 테스트 모드로 설정하고 테스트 이미지 데이터에 대하여 모델학습을 진행하고 예측을 수행, 테스트 정확도를 출력

테스트 데이터의 학습 정확도를 출력하고 마지막으로 train 데이터 정확도와 test 데이터의 정확도를 에포크 증가에 따른 그래프 변화로 출력

×
×
×

정확도 및 시연영상

예측예제



감사합니다

Q & A

×××